

RAG Systems in Production Cheat Sheet

Irurueta Docs > Guides & References > AI > RAG Systems in Production

Baseline, verified 2026-09-30: Python 3.12 · langchain 1.4.3, langchain-core 1.6.6, langgraph 1.2.12, langchain-classic 1.0.8, langgraph-checkpoint-postgres 3.1.2 · FastAPI 0.142.2 · Docling 2.131.0, Unstructured 0.27.10 · Presidio 2.2.364 · neo4j-graphrag 1.21.0, Neo4j 2026.09.0 · mcp 2.2.0, MCP spec 2026-07-28 · RAGAS 0.4.3, DeepEval 4.2.7, Open RAG Eval 0.3.0 · Spring AI 2.0.1 on Spring Boot 4.1.1, Java 21 · LangChain4j 1.20.2. Builds on Databases > Vector Databases & RAG. Verify against official docs before production use.

REFERENCE ARCHITECTURE

- Ingestion plane:** sources → parse → redact + tag (acl, hash) → chunk → embed → index writer (upsert/delete by ID, graph MERGE).
 - Query plane:** gateway (OAuth2, rate limit) → input guardrails + condense → **parallel fan-out** (vector, BM25, graph/SQL) with ACL filter → gather + RRF + rerank → generator (stream, prompt cache) → citations + grounding check.
 - Stateless orchestrator:** state lives in stores; per-retriever deadline; scale API on in-flight streams, GPU tier on tokens/s.
- Parallel retrieval: $T_{retr} = \min(\max_i T_i, \tau)$ instead of ΣT_i .

LATENCY BUDGET (P95)

$$T \approx T_{retr} + T_{rerank} + T_{TFT} + n_{out} \cdot T_{PTOT}$$

Stage	Budget
auth, guardrails, condense	150 ms
retrieval fan-out	300 ms
rerank 40 candidates	150 ms
TFT, 4k prompt tokens	800 ms
300 tokens × 20 ms	6 s (streamed)

Cut first: sequential retrieval, unconditional rerank, long prompts. TCO: $Q \cdot (c_{emb} + c_{retr} + c_{in} n_{in} + c_{out} n_{out})$ + fixed.

PARSER LADDER (CHEAPEST FIRST)

Rung	Tools	For
1 text layer	pypdf, PyMuPDF	born-digital text
2 layout	Docling, Unstructured hi_res	columns, tables, OCR
3 cloud	Textract, Azure DI, Document AI, LlamaParse	forms at scale
4 VLM	vision model to Markdown	anything; verify

Quality gate between rungs: chars/page, printable ratio, long "words". Keep page, bbox, headings, parser, ocr_confidence as metadata. JVM: TikaDocumentReader, PagePdfDocumentReader, ApacheTikaDocumentParser; layout via a Docling service.

MULTIMODAL

- Tables:** detect → extract cells → normalise (Markdown in text, JSON in metadata) → stitch across pages (same header or headerless continuation).
- Images:** VLM summary (text image) | CLIP/SigLIP shared space | ColPali late interaction on page images.
- Audio/video:** transcribe with timestamps; chunk 30–90 s; keyframes as figures.
- Visual citations:** page + bbox + signed imageUrl. Evaluate per element_type/parser.

INGESTION & FRESHNESS

- Deterministic IDs: sha256(source: chunk_index); upsert; per-document replace; job ledger + dead-letter queue.
 - Content hash skips re-embedding; source_hash, pipeline version for provenance.
 - CDC:** Debezium → Kafka (key = doc id) → re-index worker; commit offset after write; op=d + tombstone → delete chunks, cache entries, graph nodes.
 - Reconcile periodically for missed deletes; **Presidio** typed redaction at ingest (<PERSON>).
- ```
index(docs, record_manager, vs, cleanup="incremental", source_id_key="source")
langchain-classic
vectorStore.delete(b.eq("source", s).build()); vectorStore.add(chunks)
store.removeAll(metadataKey("source").isEqualTo(s)); ingestor.ingest(doc)
```

## QUERY UNDERSTANDING & ROUTING

- Condense follow-ups (skip on turn 1); rewrite; **self-query** filters ANDed with ACL (never widen); decompose (≤4 sub-questions).
- Router:** semantic (exemplars,  $\cos \geq \theta$ ) then LLM fallback: docs | sql | graph | none.
- Text-to-SQL:** read-only role, views only, single SELECT (sqlglot), allow-listed tables, LIMIT, statement\_timeout, row-level security.
- Spring AI: CompressionQueryTransformer, RewriteQueryTransformer, MultiQueryExpander; routing = a DocumentRetriever. LangChain4j: CompressingQueryTransformer, LanguageModelQueryRouter.

## INDEXING PATTERNS (INDEX ≠ RETURN)

| Pattern         | Index             | Return         |
|-----------------|-------------------|----------------|
| hypothetical Qs | questions         | chunk          |
| propositions    | atomic facts      | fact/passage   |
| auto-merging    | children          | parent if ≥p   |
| sentence window | sentence          | ±w sentences   |
| multi-vector    | summary           | raw table/code |
| RAPTOR          | summary tree      | any level      |
| contextual      | context + chunk   | chunk          |
| late chunking   | pooled token vecs | chunk          |

MultiVectorRetriever, ParentDocumentRetriever are in langchain-classic.

## MEMORY DECISION TABLE

| Kind      | Keep             | Use                  |
|-----------|------------------|----------------------|
| window    | last k msgs      | default RAG chat     |
| summary   | summary + window | long sessions        |
| long-term | user facts       | preferences; consent |

$$H_{max} = C - S - n_{ctx} - A$$

Key = verified\_sub:conversationId; do not store retrieved chunks.

```
create_agent(model, tools,
checkpointer=PostgresSaver(pool),
store=PostgresStore(pool),
context_schema=Caller,
middleware=
[SerializationMiddleware(m, trigger=
("tokens", 6000))])
MessageWindowChatMemory.builder().cha
tMemoryRepository(jdbcRepo)
.maxMessages(20).build();
a.param(ChatMemory.CONVERSATION_ID,
key)
MessageWindowChatMemory.builder().id(id)
.maxMessages(20)
.chatMemoryStore(jdbcStore).build()
// ChatMemoryProvider
```

Delete: delete\_thread, deleteByConversationId, deleteMessages. Watch memory poisoning.

## CITATIONS

- ```
{"answer": "... [1]", "grounded": true,
" Citations": [{"marker": 1,
"chunkId": "c4217", "quote": "...",
"title": "...", "url": "...",
"page": null, "start": 312,
"end": 346}]}
• IDs in the prompt (documentFormatter, metadataKeysToInclude); structured output; server resolves URLs.
• Verify: cited ID was retrieved; quote found (fuzzy ≥0.9) → spans for highlighting; track citation validity.
```

HALLUCINATION CHECKS

$$F = |supported\ claims| / |claims|$$

- LLM judge (claims + verdicts) vs **HHEM** cross-encoder (0–1, ms); Spring AI FactCheckingEvaluator.
- Modes: blocking | trailing grounding event | async sampled.
- Actions: send → retrieve again (not found) → regenerate once → drop unsupported → **abstain**.

GUARDRAIL LAYERS

- Input: Llama Guard / ShieldGemma, scope, PII, length (fail closed).
- Ingest: sanitise hidden text, zero-width chars; flag "ignore previous instructions"; trust tag.
- Context: delimit passages as data; spotlight low-trust text.
- Tools: read-only allow-list, no free egress, human approval for writes.
- Output: safety, PII, canary, grounding.
- LangChain before_agent/after_agent + PIIMiddleware; Spring SafeGuardAdvisor + CallAdvisor; LangChain4j @InputGuardrails/@OutputGuardrails (reprompt).

ACL FILTER SNIPPETS

ACL tags from the **verified token** → policy → filter, ANDed; never from the prompt; default public.

```
vs.as_retriever(search_kwargs=
{"filter":
{"$and": [{"acl": {"$in": acls}},
extra]})
a.param(VectorStoreDocumentRetriever.
FILTER_EXPRESSION,
b.in("acl", acls).build())
.dynamicFilter(q ->
metadataKey("acl").isIn(
```

```
q.metadata().invocationParameters().g
et("readableAcls")))
```

Tenancy: shared + filter | partition | index per tenant; + PostgreSQL RLS. Audit: sub, filter, retrieved/cited chunk IDs.

API CONTRACT & SSE

POST /v1/ask {question, conversationId, filters} → {answer, citations, usage, traceId, grounded}. Errors: RFC 9457.

SSE events: status → token... → citations → grounding → done | error.

```
@app.post("/v1/ask",
response_class=EventSourceResponse)
async def ask(...): yield
ServerSentEvent(event="token",
data=...)
Flux-ServerSentEvent<Object>> ...
chatClient.prompt().stream().content(
)
tokenStream.onPartialResponse(t ->
emitter.send(...)).start()
```

- 401/404 in dependencies before streaming; keep-alive comments; WebSocket for cancel.

- Idempotency-Key (per user, 24 h), 429 + Retry-After, /v1/additive, Deprecation/Sunset, CORS allow-list, async ingestion: 202 + /v1/jobs/{id}.

RAG OVER MCP

- Same retrieval core + ACL policy as REST; MCP audience ≠ API audience; no token passthrough.
- search_docs (read-only, outputSchema), docs://{component}/{page}, prompt answer_with_citations; host model writes the answer.
- Python MCPServer(token_verifier=..., auth=AuthSettings(...)); Spring @McpTool; clients: claude mcp add --transport http, VS Code .vscode/mcp.json, MultiServerMCPClient.

CACHE TIERS

Tier	Key
response	question + filters + ACL set + config_version
semantic	question embedding, $\cos \geq \theta$, same ACL set
retrieval	standalone query + filter
embedding	model + text hash

Invalidate via change events + reverse index cites: {chunk}; prompt caching: stable prefix first $c = c_{in}(n - n_c) + pn_c$; cascades $E[c] = c_s + (1 - p)c_l$.

GRAPHRAG DECISION TABLE

Question	Use
one passage answers	vector
relationships, multi-hop	vector + Cypher expansion
counts, lists, paths	text-to-Cypher (read-only)
corpus-wide themes	global search (communities)

Build: SimpleKGPipeline(schema=...), LLMGraphTransformer; resolve (exact, fuzzy); MERGE on keys; tombstone chunks; clean orphans. Retrieve: VectorCypherRetriever, HybridRetriever, Text2CypherRetriever (EXPLAIN = "r"), Neo4jText2CypherRetriever. Recs: KNN + Louvain + reader embedding.

EVALUATION METRICS

- Retrieval: recall@k, MRR, nDCG@k with UMBRELA grades 0–3.
- Answers: faithfulness, vital nugget recall R_{vital} , citation validity, answer relevance.
- Tools: RAGAS metrics.collections, DeepEval assert_test, open-rag-eval eval.
- CI gate: tolerance per metric + per-slice; pin the judge. Online: 1–5 % sampled + all flagged; A/B by user hash, two-proportion z-test.

OBSERVABILITY METRICS

- Spans: request → condense → retrieve → rerank → chat {model} → execute_tool → grounding. GenAI attrs: gen_ai.operation.name, gen_ai.request.model, gen_ai.usage.*.
- Metrics: TFTT P50/P95, end-to-end P95, tokens + cost/request, empty-retrieval rate, citation rate, abstain/handoff rate, feedback, cache hits.
- Backends: Langfuse, Phoenix, LangSmith via OTLP. Content capture off.

DEPLOY

- API + MCP (stateless) | ingestion workers | Jobs/CronJobs | GPU tier (vLLM, NIM) | managed stores; secrets manager + workload identity; timeouts > longest stream.
- config_version = prompt + model + retrieval; blue/green or canary by user hash; embedding change = shadow-index migration. DR: back up conversations, rebuild derived stores from sources.